# Tema 1

Conceptos de aplicaciones distribuidas

## 1.1. Conceptos de aplicaciones distribuidas

Una aplicación distribuida es un sistema de software cuyos componentes se ejecutan en múltiples nodos autónomos interconectados por una red y coordinan sus acciones mediante intercambio de mensajes para ofrecer un servicio unificado. Desde el punto de vista conceptual, un sistema distribuido se define como “una colección de computadores independientes que aparece ante sus usuarios como un único sistema coherente” (Tanenbaum & Van Steen, 2017). Esta definición enfatiza dos propiedades estructurales: la independencia física de los nodos y la ilusión de unidad lógica.

En términos de arquitectura de software, una aplicación distribuida desacopla responsabilidades en procesos separados que pueden ejecutarse en diferentes máquinas físicas o virtuales. Cada nodo posee su propio espacio de memoria, su propio sistema operativo y su propio reloj, lo que implica que no existe memoria compartida ni sincronización temporal perfecta; por tanto, la coordinación debe realizarse mediante protocolos de comunicación explícitos. (Coulouris et al., 2012)

### 1.1.1. Propiedades esenciales

- Concurrencia real: múltiples procesos ejecutándose simultáneamente en distintos nodos.
- Ausencia de reloj global: la sincronización se logra mediante algoritmos como relojes lógicos (Lamport).
- Fallos parciales: un nodo puede fallar mientras otros continúan operando.
- Transparencia de distribución: ocultar detalles de ubicación, migración, replicación y concurrencia al usuario final.

### 1.1.2. Modelos arquitectónicos comunes

- **Cliente-servidor:** separación funcional entre consumidores y proveedores de servicios
- **Arquitectura en capas (n-tier):** desacoplamiento de presentación, lógica de negocio y persistencia.
- **Peer-to-Peer (P2P):** nodos simétricos que comparten recursos sin jerarquía fija.
- **Microservicios**: servicios pequeños, desplegados de forma independiente y comunicados mediante API. (Newman, 2015)

El objetivo central del diseño distribuido es equilibrar rendimiento, disponibilidad y consistencia, minimizando la complejidad inherente a la comunicación remota.

## 1.2. Evolución de las aplicaciones distribuidas

La evolución de las aplicaciones distribuidas ha estado determinada por avances en redes, protocolos y modelos de programación.

1. **Centralización (Mainframes)**

En las décadas de 1960 y 1970, los sistemas eran centralizados: un único mainframe ejecutaba todas las aplicaciones. La interacción se realizaba mediante terminales “tontas”, sin capacidad de procesamiento local.

1. **Cliente-servidor**

Con la proliferación de redes locales en los años 80 y 90, emergió el modelo cliente-servidor. Este paradigma permitió distribuir la carga de procesamiento y separar la interfaz de usuario del almacenamiento de datos (Coulouris et al., 2012). Se popularizaron arquitecturas de dos y tres capas.

1. **Middleware y objetos distribuidos**

En los años 90, surgieron tecnologías como CORBA, RMI y DCOM, que abstraían la comunicación remota permitiendo invocaciones de métodos como si fueran locales. El middleware actuaba como capa intermedia que gestionaba serialización, transporte y descubrimiento de servicios.

1. **Web y servicios web**

Con la expansión de Internet, HTTP se convirtió en el protocolo dominante. Los servicios web basados en SOAP y posteriormente en REST facilitaron interoperabilidad entre plataformas heterogéneas.

1. **Cloud Computing**

La computación en la nube permitió virtualizar infraestructura y ofrecer recursos bajo demanda (Armbrust, 2010). Modelos como IaaS, PaaS y SaaS transformaron la forma en que se despliegan aplicaciones distribuidas.

1. **Microservicios y contenedores**

Actualmente, predominan arquitecturas basadas en microservicios y contenedores, lo que permite escalabilidad independiente, despliegue continuo y resiliencia ante fallos.

Esta evolución muestra un tránsito desde sistemas rígidos y centralizados hacia arquitecturas altamente dinámicas, elásticas y orientadas a servicios.

## 1.3. Principales desafíos y ventajas

El diseño de aplicaciones distribuidas implica beneficios estratégicos, pero también complejidades técnicas sustanciales.

### 1.3.1. Ventajas

- Escalabilidad horizontal: añadir nodos para aumentar capacidad.
- Alta disponibilidad: replicación de servicios y tolerancia a fallos.

- Proximidad geográfica: reducción de latencia mediante distribución regional.
- Uso eficiente de recursos: balanceo de carga.
- Flexibilidad organizacional: equipos pueden desarrollar servicios independientes.

### 1.3.2. Desafíos

- Latencia y variabilidad de red: la comunicación remota en orden de magnitud más lenta que la local.
- Consistencia de datos: mantener coherencia en presencia de replicación.
- Gestión de fallos parciales: detección y recuperación ante caídas de nodos.
- Complejidad de depuración: errores no deterministas debido a concurrencia.
- Seguridad distribuida: autenticación, autorización y cifrado en múltiples puntos.

Coulouris et al. (2012) señalan que la naturaleza asincrónica de la red y la imposibilidad de conocer el estado global exacto del sistema complican significativamente el diseño correcto.

## 1.4. Falacias de las aplicaciones distribuidas

Las falacias de la computación distribuida, identificadas por Peter Deutsch en Sun Microsystems, representan supuestos erróneos comunes que se enlistan a continuación.

- La red es confiable.
- La latencia es cero.
- El ancho de banda es infinito.
- La red es segura.
- La topología no cambia.
- Existe un solo administrador.
- El costo de transporte es cero.
- La red es homogénea.

Estas falacias evidencian que los desarrolladores tienden a extrapolar supuestos de sistemas locales al entorno distribuido. Ignorarlas puede provocar fallos catastróficos, como inconsistencias de datos, vulnerabilidades de seguridad o degradación severa del rendimiento.

## 1.5. Teorema CAP

El teorema CAP, propuesto por Eric Brewer en 2000, establece que en un sistema distribuido no es posible garantizar simultáneamente tres propiedades fundamentales:

- Consistencia (C): todas las lecturas reflejan la escritura más reciente.
- Disponibilidad (A): toda solicitud recibe una respuesta.
- Tolerancia a particiones (P): el sistema continúa funcionando pese a fallos de comunicación.

En presencia de una partición de red -evento inevitable en sistemas reales- se debe elegir entre consistencia o disponibilidad. Gilbert & Lynch (2002) demostraron formalmente esta limitación teórica.

### 1.5.1. Clasificación práctica

CP: priorizan consistencia (ej. sistemas financieros).

AP: priorizan disponibilidad (ej. sistemas de redes sociales).

CA: viables solo en ausencia de particiones (escenario teórico).

En la práctica moderna, muchos sistemas adoptan consistencia eventual, donde se acepta una inconsistencia temporal para garantizar disponibilidad y tolerancia a particiones.

## Recursos complementarios

**Martin Kleppmann-Material complementario de Designing Data-Intensive Applications**

Recurso complementario en línea: https://dataintensive.net/

https://dataintensive.net/

**Explicación técnica del teorema CAP**

Recurso complementario en línea: https://www.allthingsdistributed.com/2008/12/eventually_consistent.html

https://www.allthingsdistributed.com/2008/12/eventually_consistent.html

## Referencias


Armbrust, M. et al. (2010). A view of cloud computing. Communications of the ACM, 53(4), 50–58.

Brewer, E. (2000). Towards Robust Distributed Systems. Proceedings of PODC.

Coulouris, G.; Dollimore, J.; Kindberg, T. & Blair, G. (2012). Distributed Systems: Concepts and Design (5th ed.). Addison-Wesley.

Gilbert, S. & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. SIGACT News.

Newman, S. (2015). Building Microservices. O'Reilly Media.

Tanenbaum, A. & Van Steen, M. (2017). Distributed Systems (3rd ed.).

## Autoevaluación

**1. ¿Qué implica la ausencia de reloj global?**

a. La sincronización perfecta no existe.
b. Todos los nodos tienen la misma hora exacta.
c. No se pueden ejecutar procesos simultáneamente.
d. No se usan protocolos.

**2. ¿Cuál es un ejemplo de arquitectura distribuida moderna?**

a. Microservicios.
b. Mainframe.
c. Sistema monolítico.
d. Aplicación de escritorio aislada.

**3. Una ventaja clave es:**

a. Escalabilidad horizontal.
b. Eliminación de latencia.
c. Red sin fallos.
d. Consistencia automática.

**4. ¿Qué complica la depuración en sistemas distribuidos?**

a. Concurrencia y asincronía.
b. Exceso de memoria.
c. Interfaz gráfica.
d. Uso de bases de datos.

**5. Una falacia común es suponer que:**

a. El ancho de banda es infinito.
b. La red puede fallar.
c. Existen múltiples administradores.
d. La latencia es variable.

**6. CAP significa:**

a. Consistencia, disponibilidad, partición.
b. Control, acceso, proceso.
c. Consistencia, accesibilidad, procesamiento.
d. Capacidad, ancho, procesamiento.

**7. En una partición de red se debe elegir entre:**

a. Consistencia y disponibilidad.
b. Seguridad y latencia.
c. Hardware y software.
d. Cliente y servidor.

**8. Un sistema AP prioriza:**

a. Disponibilidad.
b. Consistencia fuerte.
c. Ausencia de red.
d. Centralización.

**9. La computación en la nube facilita:**

a. Escalabilidad bajo demanda.
b. Procesamiento aislado.
c. Eliminación de middleware.
d. Sistemas cerrados.

**10. Las falacias fueron identificadas en el contexto de:**

a. Sun Microsystems.
b. IBM.
c. Microsoft.
d. Google.

## Material complementario de la sección

- https://www.ibm.com/think/topics/distributed-systems
- https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/distributed-systems
- https://blogs.oracle.com/developers/post/fallacies-of-distributed-systems
- https://docs.oracle.com/javase/tutorial/essential/concurrency/
- https://www.baeldung.com/java-concurrency
- https://www.geeksforgeeks.org/concurrency-in-operating-system/
- https://docs.oracle.com/javase/tutorial/networking/sockets/
- https://beej.us/guide/bgnet/
- https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Client-Server_overview
- https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- https://www.rfc-editor.org/rfc/rfc6455.html
- https://socket.io/docs/v4/
- https://expressjs.com/
- https://developer.mozilla.org/en-US/docs/Web/HTTP
- https://learning.postman.com/
- Clase: Fundamentos y conceptos de las aplicaciones distribuidas Archivo — archivo incluido en la carpeta complementarios: 1. Fundamentos Conceptos AppDistrib.pdf
- Guía Laboratorio 1: Sockets (TCP-UDP) Archivo — archivo incluido en la carpeta complementarios: 1. Parc1. Laboratorio - Sockets (TCP-UDP).pdf